

Desarrollo Backend II (PBY2202)

Informe Evaluación Final Transversal – MiniMarket Plus

Índice

1. Introducción y objetivos....3
2. Planificación y alcance....4
3. Arquitectura y tecnologías....5
4. Servicios backend y operaciones....6
5. Seguridad y protección de datos....7
6. Pruebas unitarias y verificación....8
7. Documentación OpenAPI y HATEOAS....9
8. Integración y mejoras....10
9. Trazabilidad y ejecución....11
10. Conclusiones....12

1. Introducción y objetivos

El presente informe documenta la solución backend desarrollada para el caso MiniMarket Plus, cadena de minimarkets que requiere centralizar productos, inventario, usuarios, carritos y ventas. La propuesta se implementa como una aplicación Spring Boot modular, organizada por dominios y capas, que expone servicios REST protegidos y documentados. El análisis se basa en el código fuente entregado, su configuración Maven, las clases de seguridad, las pruebas automatizadas y la documentación técnica incluida.

Objetivo general

Implementar un backend mantenible para MiniMarket Plus que permita ejecutar operaciones de negocio, autenticar usuarios, autorizar acciones según roles, verificar componentes mediante pruebas unitarias y publicar una API comprensible y navegable mediante OpenAPI y HATEOAS.

Objetivos específicos

- Exponer operaciones REST para productos, categorías, inventario, carrito, usuarios, ventas y detalle de ventas.
- Aplicar autenticación JWT y autorización por roles ADMIN, CAJERO y CLIENTE.
- Separar responsabilidades mediante controladores, servicios, repositorios y entidades JPA.
- Configurar pruebas con JUnit 5, Mockito, MockMvc y Spring Security Test, además de cobertura JaCoCo.
- Describir la API con OpenAPI y enriquecer las respuestas con enlaces HATEOAS.

Alcance

La entrega corresponde a un prototipo académico integrado. Utiliza una base de datos H2 en memoria y datos iniciales para facilitar la demostración. Los módulos de dominio funcionan como servicios REST dentro de un único despliegue Spring Boot; esta decisión reduce la complejidad operativa de la EFT y conserva una separación clara que permite extraerlos a microservicios independientes en una evolución futura.

2. Planificación y alcance

El trabajo se organizó en etapas breves y verificables. La secuencia prioriza primero el modelo y las operaciones, luego la seguridad, y finalmente las pruebas y la documentación. Este orden disminuye retrabajo, porque cada capa se valida sobre una base funcional ya definida.

Etapa	Actividad principal	Resultado verificable	Responsabilidad
1. Análisis	Revisión del caso, entregables y pauta.	Alcance y criterios identificados.	Equipo
2. Backend	Entidades, repositorios, servicios y controladores.	Rutas REST y persistencia H2.	Desarrollo
3. Seguridad	JWT, BCrypt y reglas por rol.	Accesos 401/403 y permisos.	Desarrollo
4. Calidad	Pruebas unitarias, web y cobertura.	Suite de 18 métodos @Test.	Desarrollo / QA
5. Cierre	OpenAPI, HATEOAS, README e informe.	Documentación y guía de ejecución.	Equipo

Criterios de término

- La aplicación inicia con Java 17 y Spring Boot 3.4.1.
- Las rutas protegidas exigen autenticación y respetan la matriz de roles.
- Cada dominio dispone de controlador, servicio y repositorio coherentes con sus operaciones.
- La especificación se consulta en /v3/api-docs y la interfaz en /swagger-ui.html.
- Las respuestas principales incorporan relaciones HATEOAS y la suite queda disponible mediante Maven.

El informe reemplaza las capturas por descripciones técnicas paso a paso, alternativa admitida en las instrucciones. Cada evidencia señala la clase, ruta o comando que permite comprobar el componente descrito.

3. Arquitectura y tecnologías

La solución aplica una arquitectura en capas. Los controladores reciben solicitudes HTTP y construyen respuestas HATEOAS; los servicios concentran el acceso a operaciones de dominio; los repositorios extienden JpaRepository; y las entidades representan el modelo persistente. La seguridad se aplica transversalmente mediante una cadena de filtros y un filtro JWT ejecutado una vez por solicitud.

Capa / componente	Responsabilidad	Implementación
Presentación	Rutas REST, códigos HTTP y enlaces.	Controladores en com.minimarket.controller
Negocio	Contratos de operaciones por dominio.	Interfaces y clases service/impl
Persistencia	CRUD y consultas derivadas.	Spring Data JPA + repositorios
Modelo	Entidades y relaciones.	JPA: Producto, Venta, Usuario, etc.
Seguridad	Autenticación, autorización y contraseñas.	Spring Security, JWT y BCrypt
Documentación	Contrato y navegación de la API.	springdoc-openapi + HATEOAS
Calidad	Pruebas y medición de cobertura.	JUnit, Mockito, MockMvc y JaCoCo

Modelo de datos

El modelo incluye ocho entidades: Usuario y Rol se relacionan muchos a muchos; Producto pertenece a Categoria; Inventario registra movimientos asociados a Producto; Carrito vincula Usuario, Producto y cantidad; Venta pertenece a un Usuario y agrupa DetalleVenta; cada detalle referencia Producto, cantidad y precio. Las claves primarias se generan con estrategia IDENTITY y las columnas esenciales se declaran como no nulas.

Tecnologías seleccionadas

Java 17 aporta una versión estable; Spring Boot reduce configuración; JPA abstrae persistencia; H2 habilita una demostración reproducible; Maven Wrapper fija el flujo de construcción; y Lombok figura como apoyo opcional. La configuración se concentra en pom.xml y application.properties, lo que facilita localizar dependencias y parámetros.

4. Servicios backend y operaciones

Los servicios funcionales se agrupan por recurso y se exponen bajo /api. Los controladores delegan en interfaces de servicio, evitando acceder directamente a los repositorios. Para búsquedas inexistentes, las operaciones por ID responden 404; las eliminaciones correctas responden 204; y las consultas o escrituras válidas retornan el recurso o la colección correspondiente.

Servicio	Ruta base	Operaciones implementadas	Acceso
Autenticación	/api/auth	POST /login	Público
Productos	/api/productos	GET, GET/{id}, POST, PUT/{id}, DELETE/{id}	Lectura autenticada; escritura ADMIN
Categorías	/api/categorias	CRUD completo	Autenticado
Inventario	/api/inventario	CRUD de movimientos	ADMIN
Carrito	/api/carrito	CRUD y consulta del carrito	Autenticado
Usuarios	/api/usuarios	CRUD completo	ADMIN
Ventas	/api/ventas	GET, GET/{id}, POST	ADMIN o CAJERO
Detalle venta	/api/detalle-ventas	CRUD completo	Autenticado

Flujo de una operación

1. El cliente envía una solicitud HTTP con cuerpo JSON cuando corresponde.
2. Spring Security autentica la credencial y verifica el rol exigido por la ruta.
3. El controlador valida la existencia del recurso y delega la acción en el servicio.
4. La implementación del servicio utiliza el repositorio JPA para consultar o persistir.
5. El controlador devuelve estado HTTP, representación del recurso y enlaces HATEOAS.

Consultas especializadas

Además del CRUD, los contratos contemplan búsquedas por relaciones: productos por categoría, movimientos por producto, carritos y ventas por usuario, detalles por venta y usuarios por nombre. Spring Data deriva estas consultas desde métodos como findByCategoriaId, findByProductoId y findByUsername.

5. Seguridad y protección de datos

La seguridad se implementa con Spring Security mediante sesiones STATELESS. SecurityConfig desactiva CSRF para el uso REST, configura BCryptPasswordEncoder, habilita AuthenticationManager y registra JwtAuthenticationFilter antes de UsernamePasswordAuthenticationFilter. Las contraseñas de los usuarios iniciales se almacenan codificadas con BCrypt y no como texto plano en la base de datos.

Flujo de autenticación JWT

1. El usuario envía username y password a POST /api/auth/login.
2. AuthenticationManager valida las credenciales con CustomUserDetailsService y UsuarioRepository.
3. JwtUtil genera un token firmado con HMAC-SHA256 que contiene sub, roles, iat y exp.
4. La respuesta informa token, tipo Bearer, usuario y roles.
5. En solicitudes posteriores, el filtro lee Authorization: Bearer <token>, verifica firma, usuario y vencimiento, y carga las autoridades en SecurityContext.

El token posee una vigencia de cuatro horas. Si el encabezado no existe, la cadena continúa sin autenticar; si el token es inválido, el contexto se limpia y la política de acceso decide el rechazo. Esta separación evita que el controlador de negocio procese credenciales directamente.

Recurso	ADMIN	CAJERO	CLIENTE / autenticado	Sin autenticar
Productos GET	Permitido	Permitido	Permitido	401
Productos CUD	Permitido	403	403	401
Inventario	Permitido	403	403	401
Ventas GET/POST	Permitido	Permitido	403	401
Usuarios	Permitido	403	403	401
Carrito	Permitido	Permitido	Permitido	401
Categorías / detalle	Permitido	Permitido	Permitido	401

Protección y consideraciones de producción

La autorización aplica mínimo privilegio en inventario, usuarios y modificación de productos. Como endurecimiento antes de producción se recomienda externalizar la clave JWT a una variable de entorno, retirar Basic Auth y la consola H2, usar DTO para impedir la serialización del hash de contraseña, habilitar HTTPS, registrar auditoría y reemplazar H2 por un motor persistente. Estas medidas complementan el prototipo y reducen exposición de datos.

6. Pruebas unitarias y verificación

El proyecto configura spring-boot-starter-test y spring-security-test. La revisión del código identifica 18 métodos anotados con @Test distribuidos en seis clases. La estrategia combina pruebas unitarias puras, mocks de repositorio, pruebas del controlador de autenticación, carga del contexto y pruebas web con MockMvc para comprobar estados 401, 403 y 200 según el rol.

Clase de prueba	Cantidad	Aspectos verificados
AutenticacionUsuarioTest	2	Usuario existente, autoridades y excepción por usuario desconocido.
AuthControllerTest	1	Login válido, HTTP 200 y respuesta JWT Bearer.
JwtUtilTest	3	Generación, extracción de username y validación por identidad.
MinimarketApplicationTests	1	Carga completa del contexto Spring.
SeguridadControllerTest	8	Acceso a productos, inventario, ventas y usuarios por rol.
UsuarioTest	3	Creación de usuario, atributos y asignación de roles.
Total	18	Cobertura funcional, seguridad y arranque.

Evidencia textual de ejecución

1. Ubicarse en la raíz del proyecto, donde se encuentran pom.xml y Maven Wrapper.
2. Ejecutar ./mvnw clean test en macOS/Linux o .\mvnw.cmd clean test en Windows.
3. Maven limpia target, compila el código principal, compila las pruebas y ejecuta JUnit mediante Surefire.
4. El resumen debe contabilizar 18 pruebas y no registrar failures ni errors; el cierre correcto se identifica con BUILD SUCCESS.
5. JaCoCo genera el reporte HTML en target/site/jacoco/index.html para revisar cobertura por paquete, clase y método.

La suite comprueba explícitamente que una solicitud anónima a productos sea rechazada, que ADMIN pueda modificar productos y registrar inventario, que CAJERO no pueda modificar productos pero sí generar ventas, que CLIENTE no pueda registrar inventario ni ventas, y que ADMIN pueda consultar usuarios.

7. Documentación OpenAPI y HATEOAS

La API se documenta con springdoc-openapi-starter-webmvc-ui 2.8.3. OpenApiConfig define título, versión, descripción y dos esquemas de seguridad: bearerAuth para JWT y basicAuth para pruebas. Los controladores utilizan @Tag, @Operation, @ApiResponse y @SecurityRequirement para asociar cada operación con su propósito y posibles resultados.

Evidencia paso a paso de OpenAPI

1. Iniciar la aplicación con ./mvnw spring-boot:run.
2. Abrir http://localhost:8080/swagger-ui.html para visualizar recursos y operaciones agrupados.
3. Consultar http://localhost:8080/v3/api-docs para obtener el contrato OpenAPI en JSON.
4. Ejecutar POST /api/auth/login con un usuario inicial y copiar el token de la respuesta.
5. Seleccionar Authorize, ingresar Bearer <token> y probar una ruta acorde al rol.

Swagger UI permite comprobar parámetros, cuerpos, respuestas y requisitos de seguridad sin depender de una colección externa. El contrato JSON puede utilizarse para generar clientes, automatizar validaciones o integrar la API con otras herramientas.

Navegación HATEOAS

Los controladores transforman entidades en EntityModel y colecciones en CollectionModel. Los enlaces se construyen con linkTo y methodOn, de modo que las URL provienen de los propios métodos del controlador. Un producto incorpora relaciones self, productos, carrito e inventario; una venta agrega self, ventas, productos y carrito; y los demás recursos enlazan sus colecciones y dominios relacionados.

Relación	Significado para el cliente
self	Identifica la URL canónica del recurso o colección actual.
productos	Permite volver al catálogo general.
carrito	Conduce a los elementos seleccionados para compra.
inventario	Relaciona producto con movimientos de stock.
ventas / detalle-ventas	Conecta cabecera y líneas de la transacción.
crear-* / registrar-*	Expone una acción relacionada desde la colección.

Con HATEOAS, el consumidor no necesita memorizar todas las rutas: descubre transiciones válidas desde la respuesta. Esto mejora la navegabilidad solicitada por la pauta y mantiene la documentación alineada con el código.

8. Integración y mejoras

Los componentes se integran alrededor del contenedor Spring. DataInitializer crea roles, usuarios de demostración, una categoría y un producto; SecurityConfig consume CustomUserDetailsService; los controladores reciben servicios; los servicios usan repositorios; y las entidades persisten en H2. La documentación inspecciona los controladores y HATEOAS reutiliza sus métodos para formar enlaces, reduciendo discrepancias entre implementación y contrato.

Mejoras aplicadas durante el desarrollo

- **Autenticación JWT stateless** incorporada a la cadena de filtros.
- **Roles ADMIN, CAJERO y CLIENTE** con reglas centralizadas por método y ruta.
- **Contraseñas protegidas** mediante BCrypt y usuarios iniciales reproducibles.
- **Respuestas enriquecidas** con HATEOAS en todos los dominios principales.
- **Documentación OpenAPI** con esquemas de seguridad y descripciones por operación.
- **Pruebas de seguridad** con MockMvc y medición de cobertura con JaCoCo.
- **README de entrega** con comandos, credenciales, rutas y evidencias sugeridas.

Brechas y evolución recomendada

El prototipo cubre la evaluación, pero una versión productiva debe incorporar validación con Bean Validation, manejo global de excepciones, DTO de entrada y salida, paginación, trazabilidad de ventas y stock, secretos externos, base de datos persistente, migraciones y pruebas de integración adicionales. También conviene separar despliegues por dominio si el volumen o la independencia operativa justifican microservicios físicos.

9. Trazabilidad y ejecución

La siguiente matriz relaciona cada criterio técnico de la pauta con su implementación y con una evidencia textual reproducible. Su propósito es permitir que la revisión siga un camino directo desde el descriptor de evaluación hasta el archivo, ruta o comando correspondiente.

Criterio	Implementación localizada	Evidencia reproducible
Servicios y operaciones	8 controladores REST, servicios e interfaces, repositorios JPA.	Ejecutar rutas /api y revisar estados 200/204/404.
Seguridad	SecurityConfig, filtro JWT, JwtUtil, BCrypt y roles.	Login, Bearer token y pruebas de acceso 401/403.
Pruebas unitarias	6 clases y 18 métodos @Test; JaCoCo.	Ejecutar clean test y abrir target/site/jacoco/index.html.
OpenAPI y HATEOAS	OpenApiConfig, anotaciones y modelos con enlaces.	Abrir Swagger, /v3/api-docs y revisar _links.
Integración	Inicializador, inyección de servicios y persistencia H2.	Iniciar aplicación, autenticar y recorrer producto-carrito-venta.
Calidad	Paquetes por responsabilidad, interfaces y README.	Revisar estructura src/main y guía de ejecución.
Informe	Proceso, decisiones, pruebas, mejoras y conclusiones.	Documento actual en plantilla oficial.

Guía de demostración mínima

1. Iniciar el backend y comprobar que el contexto carga sin errores.
2. Autenticar admin/admin123 en /api/auth/login y conservar el JWT.
3. Listar productos y verificar los enlaces self, productos, carrito e inventario.
4. Crear o actualizar un producto como ADMIN y comprobar respuesta satisfactoria.
5. Repetir una escritura con CLIENTE y comprobar el rechazo 403.
6. Autenticar CAJERO, registrar una venta y verificar que el rol está autorizado.
7. Abrir Swagger UI y la especificación JSON para comprobar la documentación.
8. Ejecutar la suite Maven y revisar el informe de cobertura JaCoCo.

Datos de prueba

Usuario	Contraseña	Rol	Uso sugerido
admin	admin123	ADMIN	Gestión completa y configuración.
cajero	cajero123	CAJERO	Consulta y registro de ventas.
cliente	cliente123	CLIENTE	Catálogo, carrito y verificación de restricciones.

10. Conclusiones

La solución MiniMarket Plus integra en una sola aplicación los dominios principales del negocio y aplica una separación de responsabilidades comprensible. Los controladores REST, servicios y repositorios permiten gestionar productos, categorías, inventario, carrito, usuarios y ventas; mientras que las relaciones JPA sostienen la coherencia del modelo.

La seguridad constituye el eje transversal de la propuesta. La autenticación JWT, el filtro por solicitud, BCrypt y la autorización por roles distinguen claramente las capacidades de administrador, cajero y cliente. Las pruebas automatizadas declaradas verifican autenticación, tokens y permisos, y JaCoCo deja preparado el análisis de cobertura.

OpenAPI facilita comprender y ejecutar el contrato; HATEOAS transforma las respuestas en puntos de navegación entre recursos relacionados. En conjunto, estas decisiones hacen que la API sea más comprobable, mantenible y accesible para otros consumidores. La estructura modular permite evolucionar hacia persistencia productiva y despliegues independientes sin desechar la lógica ya implementada.

Aprendizajes del equipo

- La seguridad debe diseñarse junto con los casos de uso y no agregarse al final.
- Las pruebas de autorización son tan importantes como las pruebas de resultados funcionales.
- Una API documentada y navegable reduce ambigüedades de integración.
- La separación por capas simplifica el mantenimiento y la evolución del backend.

Como cierre, el proyecto satisface los ejes centrales de la actividad: operaciones backend, seguridad, pruebas, documentación técnica e integración. Las mejoras propuestas delimitan con transparencia el paso desde un prototipo académico funcional hacia una solución preparada para producción.



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.